Chapter 2

Mathematical Preliminaries

We present an overview of basic mathematical definitions used throughout the book. We assume familiarity with college-level mathematics. This chapter is far from complete but further details can be found in standard texts.

2.1 Sets

A ***set*** is a collection of distinct objects. The objects that are contained in a set, are called ***members*** or the ***elements*** of the set. The elements of a set must be distinct: a set may not contain the same element more than once. The set that contains no elements is called the ***empty set*** and is denoted by $\{\}$ or $\emptyset$.

Specification. Sets can be specified intentionally, by mathematically describing their members. For example, the set of natural numbers, traditionally written as $\mathbb{N}$, can be specified ***intentionally*** as the set of all nonnegative integral numbers. Sets can also be specified ***extensionally*** by listing their members. For example, the set $\mathbb{N} = \{0, 1, 2, \ldots\}$. We say that an element x is a *member of* A, written $x \in A$, if x is in A. More generally, sets can be specified using ***set comprehensions***, which offer a compact and precise way to define sets by mixing intentional and extensional notation.

Union and Intersection. For two sets A and B, the ***union*** $A \cup B$ is defined as the set containing all the elements of A and B. Symmetrically, their ***intersection***, $A \cap B$ is the defined as the set containing the elements that are member of both A and B. We say that A and B are ***disjoint*** if their intersection is the empty set, i.e., $A \cap B = \emptyset$.

Cartesian Product. Consider two sets A and B. The ***Cartesian product*** $A \times B$ is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$. In set notation this is

$$\{(a, b) : a \in A, b \in B\}.$$

Example 2.1. The Cartesian product of $A = \{0, 1, 2, 3\}$ and $B = \{a, b\}$ is the set of all pairings:

$$A \times B = \{(0, a), (0, b), (1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\} .$$

Set Partition. Given a set A, a partition of A is a set P of non-empty subsets of A such that each element of P is in exactly one subset in P. We refer to each element of P as a ***block*** or a ***part*** and the set P as a ***partition*** of A. More precisely, P is a partition of A if the following conditions hold:

- if $B \in P$, then $B \neq \emptyset$,
- if $A = \bigcup_{B \in P} B$, and
- if $B, C \in P$, then $B = C$ or $B \cap C = \emptyset$.

Example 2.2. If $A = \{1, 2, 3, 4, 5, 6\}$ then $P = \{\{1, 3, 5\}, \{2, 4, 6\}\}$ is a partition of A. The set $\{1, 3, 5\}$ is a block.

The set $Q = \{\{1, 3, 5, 6\}, \{2, 4, 6\}\}$ is a not partition of A, because the element 6 is contained in any of the blocks.

2.2 Relations and Functions

A ***(binary)*** **relation** ***from a set*** A ***to set*** B is a subset of the Cartesian product of A and B. For a relation $R \subseteq A \times B$, the set $\{a : (a, b) \in R\}$ is referred to as the ***domain*** of R, and the set $\{b : (a, b) \in R\}$ is referred to as the ***range*** of R.

A ***mapping from*** A ***to*** B is a relation $R \subset A \times B$ such that $|R| = |\text{domain}(R)|$, i.e., for every a in the domain of R there is only one b such that $(a, b) \in R$. A mapping is also called a ***function***.

Example 2.3. Consider the sets $A = \{0, 1, 2, 3\}$ and $B = \{a, b\}$.

The set:

$$X = \{(0, a), (0, b), (1, b), (3, a)\}$$

is a relation from A to B since $X \subset A \times B$, but not a mapping (function) since 0 is repeated.

The set

$$Y = \{(0, a), (1, b), (3, a)\}$$

is both a relation and a function from A to B since each element only appears once on the left.

The domain of Y is $\{0, 1, 3\}$ and the range is $\{a, b\}$. It is, however, not a sequence since there is a gap in the domain.

2.3 Graph Theory

2.3.1 Basic Definitions

Definition 2.4. A ***directed graph*** or (***digraph***) is a pair $G = (V, A)$ where

- V is a set of ***vertices***, and
- $A \subseteq V \times V$ is a set of ***directed edges*** or ***arcs***.

In a digraph, each arc is an ordered pair $e = (u, v)$. A digraph can have ***self loops*** (u, u). Example 2.6 shows a digraph with 4 vertices and 4 arcs.

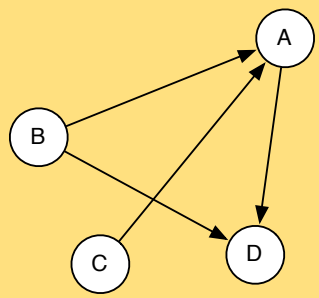
Definition 2.5. [Undirected graph] An ***undirected graph*** is a pair $G = (V, E)$ where

- V is a set of ***vertices*** (or nodes), and
- $E \subseteq \binom{V}{2}$ is a set of edges.

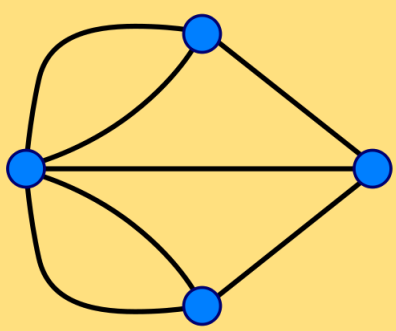
In an undirected graph, each edge is an unordered pair $e = \{u, v\}$ (or equivalently $\{v, u\}$). By this definition an undirected graph cannot have self loops since $\{v, v\} = \{v\} \notin \binom{V}{2}$.

While directed graphs represent possibly asymmetric relationships (my web page points to yours, but yours does not necessarily point back), undirected graphs represent symmetric relationships. Directed graphs are in some sense more general than undirected graphs since we can easily represent an undirected graph by a directed graph by replacing an edge with two arcs, one in each direction. Indeed, this is usually the way we represent undirected graphs in data structures.

Example 2.6. An example directed graph with 4 vertices:



An undirected graph on 4 vertices, representing the Königsberg problem. (Picture Source: Wikipedia):



Graphs come with a lot of terminology, but fortunately most of it is intuitive once we understand the concept. In this section, we consider graphs that do not have any data associated with edges, such as weights. In the next section and more comprehensively in Chapter 16, we consider weighted graphs, where the weights on edges can represent a distance, a capacity or the strength of the connection.

Neighbors. A vertex u is a ***neighbor*** of, or equivalently ***adjacent*** to, a vertex v in a graph $G = (V, E)$ if there is an edge $\{u, v\} \in E$. For a directed graph a vertex u is an ***in-neighbor*** of a vertex v if $(u, v) \in E$ and an ***out-neighbor*** if $(v, u) \in E$. We also say two edges or arcs are neighbors if they share a vertex.

Neighborhood. For an undirected graph $G = (V, E)$, the ***neighborhood*** $N_G(v)$ of a vertex $v \in V$ is its set of all neighbors of v, i.e., $N_G(v) = \{u \mid \{u, v\} \in E\}$. For a directed graph we use $N_G^+(v)$ to indicate the set of out-neighbors and $N_G^-(v)$ to indicate the set of in-neighbors of v. If we use $N_G(v)$ for a directed graph, we mean the out neighbors. The neighborhood of a set of vertices $U \subseteq V$ is the union of their neighborhoods, e.g. $N_G(U) = \bigcup_{u \in U} N_G(y)$, or $N_G^+(U) = \bigcup_{u \in U} N_G^+(u)$.

Incidence. We say an edge is ***incident*** on a vertex if the vertex is one of its endpoints. Similarly we say a vertex is incident on an edge if it is one of the endpoints of the edge.

Degree. The ***degree*** $d_G(v)$ of a vertex $v \in V$ in a graph $G = (V, E)$ is the size of the neighborhood $|N_G(v)|$. For directed graphs we use ***in-degree*** $d_G^-(v) = |N_G^-(v)|$ and ***out-degree*** $d_G^+(v) = |N_G^+(v)|$. We will drop the subscript G when it is clear from the context which graph we're talking about.

Paths. A ***path*** in a graph is a sequence of adjacent vertices. More formally for a graph $G = (V, E)$, $Paths(G) = \{P \in V^+ \mid 1 \leq i < |P|, (P_i, P_{i+1}) \in E\}$ is the set of all paths in G, where V^+ indicates all positive length sequences of vertices (allowing for repeats). The length of a path is one less than the number of vertices in the path—i.e., it is the number of edges in the path. A path in a finite graph can have infinite length. A ***simple path*** is a path with no repeated vertices. Please see the remark below, however.

Remark 2.7. Some authors use the terms walk for path, and path for simple path. Even in this book when it is clear from the context we will sometimes drop the "simple" from simple path.

Reachability and connectivity. A vertex v is ***reachable*** from a vertex u in G if there is a path starting at v and ending at u in G. We use $R_G(v)$ to indicate the set of all vertices reachable from v in G. An undirected graph is ***connected*** if all vertices are reachable from all other vertices. A directed graph is ***strongly connected*** if all vertices are reachable from all other vertices.

Cycles. In a directed graph a ***cycle*** is a path that starts and ends at the same vertex. A cycle can have length one (i.e. a ***self loop***). A ***simple cycle*** is a cycle that has no repeated vertices other than the start and end vertices being the same. In an undirected graph a (simple) ***cycle*** is a path that starts and ends at the same vertex, has no repeated vertices other than the first and last, and has length at least three. In this course we will exclusively talk about simple cycles and hence, as with paths, we will often drop simple.

Exercise 2.8. Why is important in a undirected graph to require that a cycle has length at least three? Why is important that we do not allow repeated vertices?

Trees and forests. An undirected graph with no cycles is a ***forest***. A forest that is connected is a ***tree***. A directed graph is a forest (or tree) if when all edges are converted to undirected edges it is undirected forest (or tree). A ***rooted tree*** is a tree with one vertex designated as the root. For a directed graph the edges are typically all directed toward the root or away from the root.

Directed acyclic graphs. A directed graph with no cycles is a ***directed acyclic graph*** (DAG).

Distance. The ***distance*** $\delta_G(u, v)$ from a vertex u to a vertex v in a graph G is the shortest path (minimum number of edges) from u to v. It is also referred to as the ***shortest path length*** from u to v.

Diameter. The ***diameter*** of a graph G is the maximum shortest path length over all pairs of vertices in G, i.e., $\max\{\delta_G(u, v) : u, v \in V\}$.

Multigraphs. Sometimes graphs allow multiple edges between the same pair of vertices, called ***multi-edges***. Graphs with multi-edges are called ***multi-graphs***. We will allow multi-edges in a couple algorithms just for convenience.

Sparse and dense graphs. By convention we will use the following definitions:

$$\begin{aligned} n &= |V| \\ m &= |E| \end{aligned}$$

Note that a directed graph can have at most n^2 edges (including self loops) and an undirected graph at most $n(n-1)/2$. We informally say that a graph is ***sparse*** if $m \ll n^2$ and ***dense*** otherwise. In most applications graphs are very sparse, often with only a handful of neighbors per vertex when averaged across vertices, although some vertices could have high degree. Therefore, the emphasis in the design of graph algorithms, at least for this book, is typically on algorithms that work well for sparse graphs.

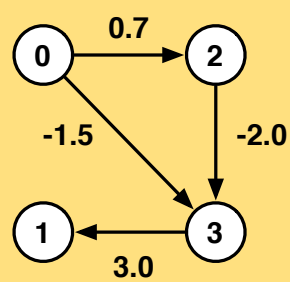
2.3.2 Weighted Graphs

Many applications of graphs require associating weights or other values with the edges of a graph. Such graphs can be defined as follows.

Definition 2.9. [Weighted and Edge-Labeled Graphs] An *edge-labeled graph* or a *weighted graph* is a triple $G = (E, V, w)$ where $w : E \to L$ is a function mapping edges or directed edges to their labels (weights) , and L is the set of possible labels (weights).

In a graph, if the data associated with the edges are real numbers, we often use the term "weight" to refer to the edge labels, and use the term "weighted graph" to refer to the graph. In the general case, we use the terms "edge label" and edge-labeled graph. Weights or other values on edges could represent many things, such as a distance, or a capacity, or the strength of a relationship.

Example 2.10. An example directed weighted graph.



As it may be expected, basic terminology on graphs defined above straightforwardly extend to weighted graphs.

2.3.3 Subgraphs

When working with graphs, we sometimes wish to refer to parts of a graph. To this end, we can use the notion of a subgraph, which refers to a graph contained in a larger graph. A subgraph can be defined as any subsets of edges and vertices that together constitute a well defined graph.

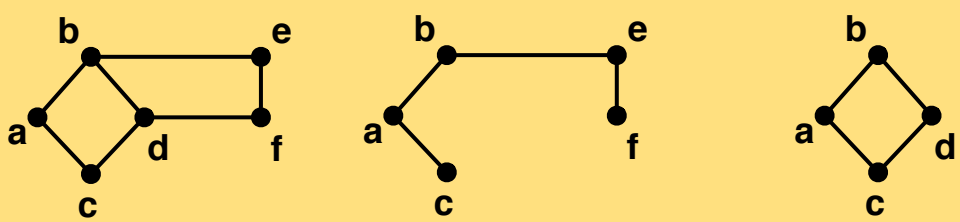
Definition 2.11. [Subgraph] Let $G = (V, E)$ and $H = (V', E')$ be two graphs. H is a subgraph of if $V' \subseteq V$ and $E' \subseteq E$.

Note that since H is a graph, the vertices defining each edge are in the vertex set V', i.e., for an undirected graph $E' \subseteq \binom{V'}{2}$). There are many possible subgraphs of a graph.

An important class of subgraphs are ***vertex-induced subgraphs***, which are maximal subgraphs defined by a set of vertices. A vertex-induced subgraph is maximal in the sense that it contains all the edges that it can possibly contain. In general when an object is said to be a ***maximal*** "X", it means that nothing more can be added to the object without violating the property "X".

Definition 2.12. [Vertex-Induced Subgraph] The subgraph of $G = (V, E)$ induced by $V' \subseteq V$ is the graph $H = (V', E')$ where $E' = \{\{u, v\} \in E \mid u \in V', v \in V'\}$.

Example 2.13. Some vertex induced subgraphs:



Original graph Induced by $\{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{e}, \mathtt{f}\}$ Induced by $\{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}\}$

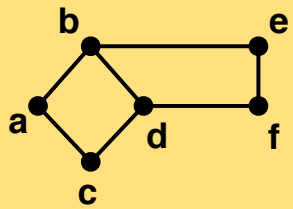
Although we will not use it in this book, it is also possible to define an induced subgraph in terms of a set of edges by including in the graph all the vertices incident on the edges.

Definition 2.14. [Edge-Induced Subgraph] The subgraph of $G = (V, E)$ induced by $E' \subseteq E$ is a graph $H = (V', E')$ where $V' = \cup_{e \in E} e$.

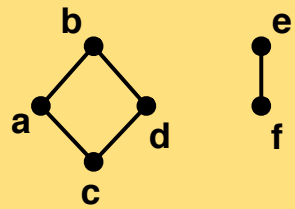
2.3.4 Connectivity and Connected Components

Recall that in a graph (either directed or undirected) a vertex v is reachable from a vertex u if there is a path from u to v. Also recall that an undirected graph is connected if all vertices are reachable from all other vertices.

Example 2.15. Two example graphs shown. The first in connected; the second is not.



Graph 1



Graph 2

An important subgraph of an undirected graph is a ***connected component*** of a graph.

Definition 2.16. [Connected Component] Let $G = (V, E)$ be an undirected graph. A subgraph H of G is a connected component of G if it is a maximally connected subgraph of G.

Here, "maximally connected component" means we cannot add any more vertices and edges from G to H without disconnecting H. In the graphs shown in Example 2.15, the first graph has one connected component (hence it is connected); the second has two connected components.

Using vertex-induced subgraphs, we can specify a connected component of a graph by simply specifying the vertices in the component.

Example 2.17. Connected components of our second example graph Example 2.15 can be specified as $\{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}\}$ and $\{\mathtt{e}, \mathtt{f}\}$.

2.3.5 Graph Partition

Recall that a partition of a set A is a set P of non-empty subsets of A such that each element of A is in exactly one subset, also called block, $B \in P$. We define a ***partition of a graph*** as a partition of its vertex set. More precisely, given graph $G = (V, E)$, we define a partition of G as a set of graphs $P = \{G_1 = (V_1, E_1) \ldots G_k = (V_k, E_k)\}$, where $\{V_1, \ldots, V_k\}$ is a (set) partition of V and $G_1, \ldots, G_k$ are vertex-induced subgraphs of G with respect to $V_1, \ldots, V_k$ respectively. As in set partitioning, we use the term ***part*** or ***block*** to refer to each vertex-induced subgraph $G_1, \ldots, G_k$.

In a graph partition, we can distinguish between two kinds of edges: internal edges and cut edges. ***Internal edges*** are edges that are within a block; ***cut edges*** are edges that are between blocks. One way to partition a graph is to make each connected component a block. In such a partition, there are no cut edges between the partitions.

2.3.6 Trees

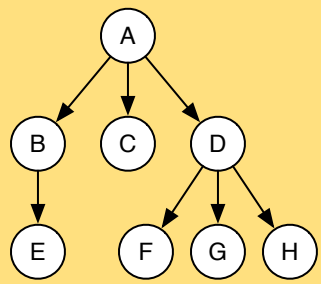
An undirected graph is a tree if it does not have cycles and it is connected. A ***rooted tree*** is a tree with a distinguished root node that can be used to access all other nodes (Definition 2.18). An example of a rooted tree along with the associated terminology is given in Example 2.19.

Definition 2.18. [Rooted Tree] A ***rooted tree*** is a directed graph such that

1. One of the vertices is the ***root*** and it has no in edges.
2. All other vertices have one in-edge.
3. There is a path from the root to all other vertices.

By convention we use the term ***node*** instead of vertex to refer to the vertices of a rooted tree. A node is a ***leaf*** if it has no out edges, and an ***internal node*** otherwise. For each directed edge (u, v), u is the ***parent*** of v, and v is a ***child*** of u. For each path from u to v (including the empty path with $u = v$), u is an ***ancestor*** of v, and v is a ***descendant*** of u. For a vertex v, its ***depth*** is the length of the path from the root to v and its ***height*** is the longest path from v to any leaf. The ***height of a tree*** is the height of its root. For any node v in a tree, the ***subtree rooted at*** v is the rooted tree defined by taking the induced subgraph of all vertices reachable from v (i.e. the vertices and the directed edges between them), and making v the root. As with graphs, an ***ordered rooted tree*** is a rooted tree in which the out edges (children) of each node are ordered.

Example 2.19. An example of a rooted tree:



root	:	A
leaves	:	E, C, F, G, and H
internal nodes	:	A, B, and D
children of A	:	B, C and D
parent of E	:	B
descendants of A	:	all nodes, including A itself
ancestors of F	:	F, D and A
depth of F	:	2
height of B	:	1
height of the tree	:	2
subtree rooted at D	:	the rooted tree consisting of D, F, G and H

.